Marinescu Alexandra-Gabriela 334CD

News Aggregator

1. Feedback:

Implementarea temei a durat aproximativ 3 zile.

În ceea ce privește îmbunătățirea implementării temei, dacă ar fi sa o fac din nou, aș încerca să transmit prin referintă obiectele și structurile de date de care am nevoie, astfel încât să fie mai mult OOP-orientat și să se respecte toate principiile SOLID. Nu aș mai declara toate variabilele și metodele statice. Aș încerca să nu mai folosesc așa des barieră. În schimb, am încercat să modularizez functionalitătile temei, să fac metode generale și mai multe clase care sa mă ajute la implementare, să nu fie toată logica codului într-o singură clasă.

Enunțul temei a fost unul clar și concis. Un detaliu destul de important a fost omis (ex. nu se specifica în cazul campului "category" ca pot exista duplicate, fiind un număr mare de articole sesizarea acestui aspect a fost costisitor din punct de vedere al timpului petrecut în debug). În schimb, ceea ce a ajutat foarte mult în înțelegerea cerintei, a fost exemplul de rulare și explicația rezultatelor obtinute.

Mi-a placut ca a fost o tema care a necesitat mult lucru cu structuri de tipul concurrent, astfel le-am înțeles mai bine.

2. Strategia de paralelizare

2.1. Secvențial:

- Citesc din fișierele de input și rețin categoriile, limbile, cuvintele de interes cu care trebuie sa lucrez.

2.2. În paralel:

- Threadurile procesează articolele și extrag campurile de interes din formatul JSON.

–Bariera–

2.3. Secvențial (din thread-ul cu id-ul 0)

- Elimin articolele duplicate în funcție de uuid și title.

–Bariera–

2.4. În paralel:

- Extrag în structura *language* toate articolele ale căror limbi se găsesc în fișierul de input *languages.txt.*
- Pentru articolele scrise în limba engleză: număr în câte articole apar cuvinte de interes.
- Număr cate articole sunt scrise în fiecare limbă.
- Extrag în structura *categoriesMap* toate articolele ale căror categorii se regasesc in fișierul *categories.txt.*
- Număr cate articole aparțin categoriilor specificate în *categories.txt.*
- Număr cate articole sunt scrise de fiecare autor care apare.
- Rețin în *processedArticles,* toate articolele procesate (fără duplicate), cheia fiind uuid, iar valoarea published, valori necesare pentru afișarea finală în fișierul *all_articles.txt.*

–Barieră–

2.5. Secvențial (din thread-ul cu id-ul 0)

- Am creat fișierele de output
- Am ordonat lexicografic categoriile.
- Am ordonat lexicografic limbile după uuid.
- Am ordonat descrescător după published, fișierul *all_articles*, iar pentru egalitate am ordonat crescător după uuid.
- Am ordonat cuvintele de interes.
- Am scris rezultatele în fișierele de output.

2.2. (mai multe explicații)

Fiecare thread procesează un număr de articole, acest număr specific fiecărui thread este determinat prin calcularea indecsilor start și end, în funcție de id-ul threadului. Thread-urile mapează conținutul JSON la campurile de interes (uuid, title, categories, text, language, author, url, published) în obiecte *Article*. Am folosit record pentru obiectul *Article* in loc de class, deoarece toate variabilele sunt de tip final, acestea nu o sa fie modificate de-a lungul executarii si nu trebuie definită metoda get() pentru accesarea valorilor. Acest lucru am făcut în metoda *parseUniqueArt(String file)*, din clasa *NewsAgregator.* Tot în această metodă, verific daca titlul articolului curent a fost deja procesat. Folosesc structura *differentTitle*, care este un *ConcurrentHashMap<String, Article>* unde cheia este titlul, iar valoarea este obiectul *Article* asociat acelui titlu. Folosesc metoda *putIfAbsent* pentru a verifica dacă deja exista cheia în map (dacă există titlul), în caz afirmativ, incrementez variabila *atomicTitleCount* care este de tip *AtomicInteger.* Am folosit *AtomicInteger* pentru a nu fi permis cat mai multe threaduri să scrie în același timp. Am folosit *ConcurrentHashMap*, deoarece lock-ul este pus intern pe cate un bucket, acest lucru permitand mai multor thread-uri sa adauge valori în map în mod paralel, la hash-uri

diferite. Repet logica pentru uuid, folosind *differentUuidMap* și contorul *AtomicInteger atomicUuidCount.*

2.3. (mai multe explicații)

În metoda *writeFinalResults()* din clasa *NewsAgregator* elimin duplicatele, folosindu-mă de datele obținute la metoda *parseUniqueArt,* explicată mai sus. Bariera este folosită aici (intre apelul metodei *parseUniqueArt* și *writeFinalResults* în metoda *run()* din clasa *MyThread*) pentru a mă asigura că toate articolele au fost procesate de către toate thread-urile, pentru a asigura eliminarea corecta si completa a duplicatelor. Dacă nu era folosită bariera, riscam să nu elimin toate duplicatele. O altă barieră este folosită după eliminarea duplicatelor, pentru a mă asigura că celelalte threaduri nu au început alte operații de procesare a datelor înainte sa se termine operația de eliminare a duplicatelor. Articolele fără duplicate sunt retinute în *Article[] articlesArray.*

2.4. (mai multe explicatii)

Înainte de a trece la următoarele operații, trebuie să recalculez indecsii de start și end pentru fiecare thread, intrucât dimensiunea problemei s-a schimbat după eliminarea duplicatelor. Dimensiunea problemei este reprezentată acum de numărul de articole singulare. În mod paralel, salvez uuid-urile articolelor ale căror limbi se află în fișierul de input *languages.txt.* Fac acest lucru utilizand *ConcurrentHashMap<String, ConcurrentLinkedQueue<String>> languages,* în metoda *organizeByLanguage(Article article)*. Cheia este limba, iar valoarea este un queue thread-safe in care rețin uuid-urile articolelor cu limba respectivă. Folosesc *ConcurrentLinkedQueue*, deoarece se pot efectua mai repede mai multe operații de scriere în paralel. În mod asemănător organizez și pe categorii. Pentru a determina best author, top Language, numărul de cuvinte de interes, folosesc *ConcurrentHashMap<String, AtomicInteger>* pentru a permite mai multor threaduri să introducă o cheie nouă cu o valoare, în același timp. *AtomicInteger* este folosit pentru a nu lăsa mai multe thread-uri sa modifice contorul în același timp. După aceste operații, folosesc o barieră pentru a mă asigura ca toate datele au fost procesate înainte de a începe scrierea in fișierele de output.

3. Analiza de performanta și scalabilitate

- Setup de testare:

Dimensiunea dataset-ului de test este 13789 (numarul total de articole).
CPU: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
Număr de nuclee: 4 nuclee fizice
RAM: 4GB (WSL)
SO: Linux
Versiune Java: OpenJDK 25.0.1

- Rezultate:

Setul de date de intrare este testul 5.

Rulare 1:

	SpeedUp: S(P) = T(1) / T(P)	Eficienta: E(P) = S(P) / P	Timpul de executie(s)
P = 1	1	1	49.63
P = 2	1.69	0.84	29.31
P = 3	2.44	0.81	20.29
P = 4	2.35	0.58	21.04

Rulare 2:

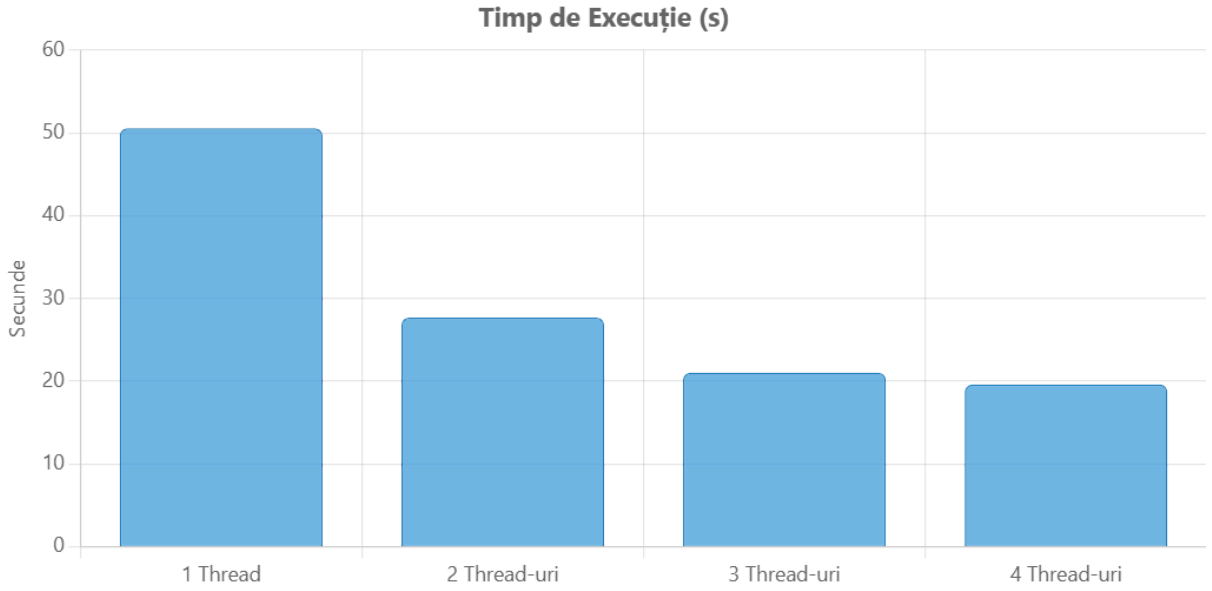
	SpeedUp: S(P) = T(1) / T(P)	Eficienta: E(P) = S(P) / P	Timpul de executie(s)
P = 1	1	1	51.50
P = 2	1.83	0.91	28.04
P = 3	2.61	0.87	19.73
P = 4	2.60	0.65	19.79

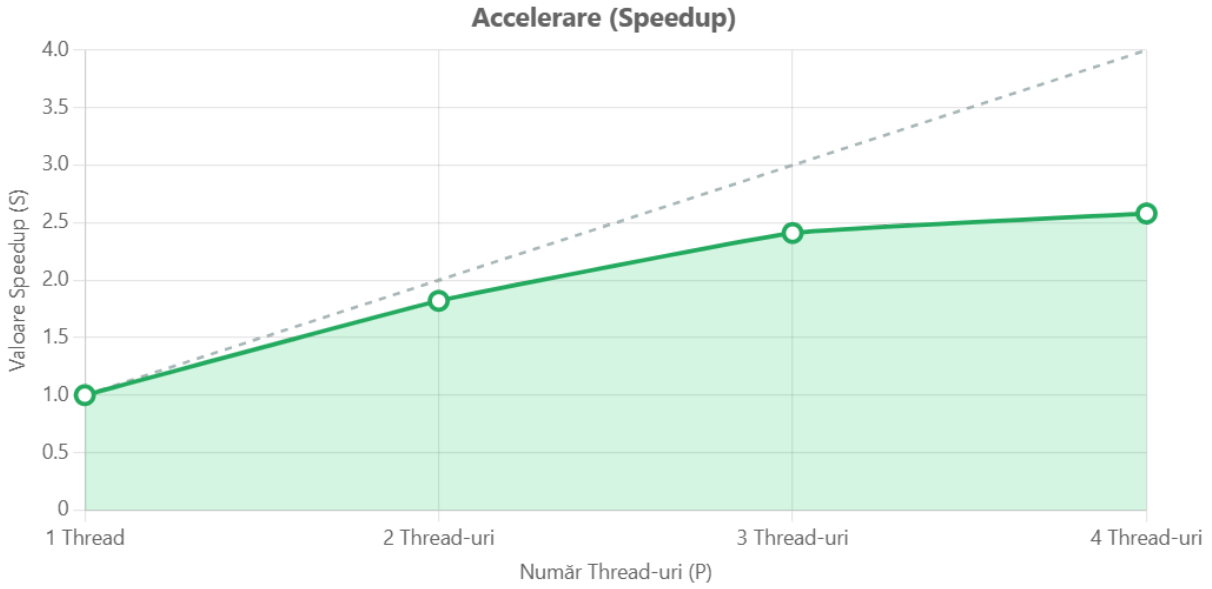
Rulare 3:

	SpeedUp: S(P) = T(1) / T(P)	Eficienta: E(P) = S(P) / P	Timpul de executie(s)
P = 1	1	1	50.58
P = 2	1.96	0.98	25.74
P = 3	2.18	0.72	23.10
P = 4	2.81	0.70	17.99

Timp mediu de execuție pentru 1 thread (3 rulări) : 50.57
Timp mediu de execuție pentru 2 threads (3 rulări) : 27.69
Timp mediu de execuție pentru 3 thread (3 rulări) : 21.04
Timp mediu de execuție pentru 4 thread (3 rulări) : 19.60

Speed-up mediu pentru 2 threads : 1.82
Speed-up mediu pentru 3 threads : 2.41
Speed-up mediu pentru 4 threads : 2.58





- Analiza și concluzii:

Am observat că timpul de execuție scade pe măsură ce numărul de fire de thread-uri crește. Cea mai mare creștere de performantă se înregistrează la trecerea de la execuția secvențială la 2 thread-uri, unde obțin un speedup de 1.82 și o eficiență de 91%. Acest lucru demonstrează că algoritmul paralelizează eficient sarcinile de parsare și procesare a textului.

Stabilizarea se intampla la threadurile 3-4. Odată cu creșterea numărului de thread-uri la 4, curba de accelerare începe să se aplatizeze. Deși timpul scade la 21,04 (media), eficiența scade la 64.5%.

Limitarea I/O (Input/Output Bottleneck): Codul este intensiv în operații de citire și scriere pe disc (parsarea fișierelor JSON mari și scrierea multor fișiere text mici la final), aceste operații fiind CPU intensive.

Schimbarea de context între thread-uri adaugă overhead, cand efectuez secvențial niste operații doar din thread-ul 0.

Am latenta intrucat folosesc ConcurrentLinkedQueue și ConcurrentHashMap care intern folosesc Volatile, se efectuează operații de citire și scriere direct pe disc, nu se foloseste memoria cache pentru un acces rapid al datelor.